Name: Muhammad Usama Bilal

Roll Number :CSI-B1-460

INSTRUCTOR: MUHAMMAD SAAD

Date: 23-4-206

Task Overview: Automation Basics & Environment Setup

- **Description :**

In this task, you will explore how automation is used in cybersecurity to handle repetitive processes such as scanning networks and analyzing logs. You will also build simple scripts that perform these tasks automatically.

- **Technical Objectives**

- Understand why scripting is important in modern cybersecurity environments
- Set up Python and Bash environments in your system
- Study common libraries like **Scapy, Requests, Subprocess**
- Create:
  - A **Bash script** for network scanning
  - A **Python script** for log analysis
- Use Scapy to generate network packets for testing

- **DAY 1 – Understanding Automation**

What is Automation in Cybersecurity?

Automation means using scripts or tools to perform tasks like scanning, monitoring, and analysis without manual effort.

Importance in SOC (Security Operations Center)

- Speeds up threat detection
- Reduces human mistakes
- Handles large-scale monitoring

Importance in Perple Team

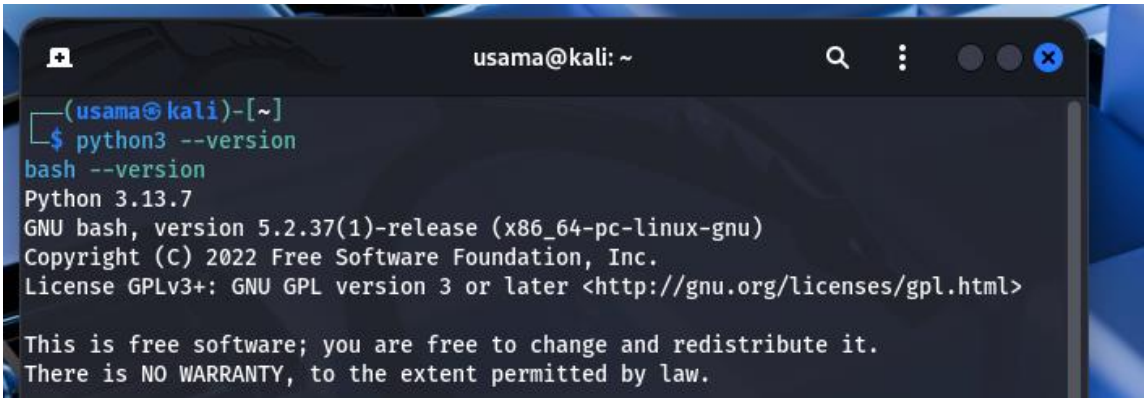
- Automates scanning and attacks
- Helps in creating payloads
- Saves time during penetration testing

Example Tools

Tool	Function	Language	Automation Role
Nmap	Scan networks	C/C++	Finds hosts & ports
Wireshark	Analyze traffic	C	Captures packets
Metasploit	Exploitation	Ruby	Automates attacks

➢ DAY 2 – Setup Environment

Kali Linux Terminal



Install Required Libraries:

```
┌──(myenv)─(usama㉿kali)-[~]
└─$ pip install scapy
pip install requests
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl.metadata (5.8 kB)
Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 747.5 kB/s  0:00:03
Installing collected packages: scapy
Successfully installed scapy-2.7.0
Collecting requests
  Downloading requests-2.33.1-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.13-py3-none-any.whl.metadata (8.0 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.4.22-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.33.1-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (215 kB)
Downloading idna-3.13-py3-none-any.whl (68 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading certifi-2026.4.22-py3-none-any.whl (135 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.4.22 charset_normalizer-3.4.7 idna-3.13 requests-2.33.1 urllib3-2.6.3
```

- **Scapy** → Used for creating and sending packets
- **Requests** → Used for sending HTTP requests

➢ DAY 3 – Tool Study

```
┌──(myenv)─(usama㉿kali)-[~]
└─$ nmap -h
wireshark
msfconsole
Nmap 7.95 ( https://nmap.org )
Usage: nmap [Scan Type(s)] [Options] {target specification}
TARGET SPECIFICATION:
  Can pass hostnames, IP addresses, networks, etc.
  Ex: scanme.nmap.org, microsoft.com/24, 192.168.0.1; 10.0.0-255.1-254
  -iL <inputfilename>: Input from list of hosts/networks
  -iR <num hosts>: Choose random targets
  --exclude <host1[,host2][,host3],...>: Exclude hosts/networks
  --excludefile <exclude_file>: Exclude list from file
HOST DISCOVERY:
  -sL: List Scan - simply list targets to scan
  -sn: Ping Scan - disable port scan
  -Pn: Treat all hosts as online -- skip host discovery
  -PS/PA/PU/PY[portlist]: TCP SYN, TCP ACK, UDP or SCTP discovery to given ports
  -PE/PP/PM: ICMP echo, timestamp, and netmask request discovery probes
  -PO[protocol list]: IP Protocol Ping
  -n/-R: Never do DNS resolution/Always resolve [default: sometimes]
  --dns-servers <serv1[,serv2],...>: Specify custom DNS servers
  --system-dns: Use OS's DNS resolver
  --traceroute: Trace hop path to each host
SCAN TECHNIQUES:
  -sS/sT/sA/sW/sM: TCP SYN/Connect()/ACK/Window/Maimon scans
  -sU: UDP Scan
  -sN/sF/sX: TCP Null, FIN, and Xmas scans
  --scanflags <flags>: Customize TCP scan flags
  -sI <zombie host[:probeport]>: Idle scan
  -sY/sZ: SCTP INIT/COOKIE-ECHO scans
  -sO: IP protocol scan
  -b <FTP relay host>: FTP bounce scan
PORT SPECIFICATION AND SCAN ORDER:
  -p <port ranges>: Only scan specified ports
    Ex: -p22; -p1-65535; -p U:53,111,137,T:21-25,80,139,8080,S:9
  --exclude-ports <port ranges>: Exclude the specified ports from scanning
  -F: Fast mode - Scan fewer ports than the default scan
  -r: Scan ports sequentially - don't randomize
  --top-ports <number>: Scan <number> most common ports
  --port-ratio <ratio>: Scan ports more common than <ratio>
```

Example (Nmap)

- Automates → Network scanning
- Built in → C/C++
- Uses → Networking libraries

➢ DAY 4 – Bash Automation Script

Step 1: Create Script (Terminal)

```
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                                          scanner.sh *
#!/bin/bash

read -p "Subnet enter karo (example 192.168.1.0/24): " subnet

output="result.txt"
> "$output"

echo "Scanning started..."

for host in $(nmap -sn "$subnet" | grep "Nmap scan report" | cut -d " " -f 5)
do
    echo "Live Host: $host" | tee -a "$output"

    ports=$(nmap -p 22,80 --open "$host" | grep "/tcp" | cut -d "/" -f 1 | paste -sd ",")

    echo "Open Ports: $ports" | tee -a "$output"
    echo "-----------------" | tee -a "$output"
done

echo "Done! Results saved in result.txt"
```

Output:

```
┌──(kali㉿kali)-[~]
└─$ nano scanner.sh

┌──(kali㉿kali)-[~]
└─$ chmod +x scanner.sh

┌──(kali㉿kali)-[~]
└─$ ./scanner.sh
Subnet enter karo (example 192.168.1.0/24): 192.168.1.0/24
Scanning started...
Live Host: 192.168.1.0
Open Ports:
-----------------
Live Host: 192.168.1.1
Open Ports:
-----------------
Live Host: 192.168.1.2
Open Ports:
-----------------
Live Host: 192.168.1.3
Open Ports:
-----------------
```

Key Implementation Points

- **Automation Approach:**
  Network reconnaissance was automated using a custom Bash script (scanner.sh), reducing the need for manual scanning.
- **Scanning Methodology:**
  A two-phase process was applied — first discovering active hosts using nmap -sn, followed by checking selected ports (22 and 80) on those live systems.
- **Data Processing:**
  Commands like grep, cut, and tee were utilized to extract relevant information and store it in a structured and readable format inside result.txt.
- **Working Environment:**
  Tasks were performed in the Kali Linux terminal, including managing script execution permissions using chmod +x.

Python Scripting :

```
  GNU nano 8.6
import re

sqli_patterns = [
    r"(\%27)|(\')|(\-\-)|(\%23)|(#)",
    r"((\%3D)|(=))[^\n]*((\%27)|(\')|(\-\-)|(\%3B)|(;))",
    r"\w*((\%27)|(\'))(\s*)((\%6F)|o|(\%4F))((\%72)|r|(\%52))",
    r"((\%27)|(\'))union",
]

def is_sqli(line):
    line_lower = line.lower()
    return any(re.search(p, line_lower) for p in sqli_patterns)

def extract_ip(line):
    match = re.match(r"(\d+\.\d+\.\d+\.\d+)", line)
    return match.group(1) if match else None

def analyze_log(file_path, subnet="10.0.2."):
    unique_ips = set()
    suspicious = []

    with open(file_path, "r", errors="ignore") as f:
        for line in f:
            ip = extract_ip(line)
            if ip and ip.startswith(subnet):
                unique_ips.add(ip)

                if is_sqli(line):
                    suspicious.append((ip, line.strip()))

    return unique_ips, suspicious
```

Output:

```
┌──(usama㉿usama)-[~]
└─$ python3 log_analyzer.py
Unique IPs Found: 4
Successfully installed scapy-2.7.0
Suspicious Requests (Local subnet only):
10.0.2.15 → 10.0.2.15 - GET /login.php?id=' OR '1'='1
10.0.2.25 → 10.0.2.25 - GET /data.php?id=UNION SELECT *
```

Project Overview

- **Objective:** Designed and implemented a Python-based Security Log Analyzer (log_analyzer.py) to monitor and detect web-based threats from server log files (access.log).
- **Threat Detection Approach:** Utilized Regular Expressions (Regex) to identify well-known SQL Injection (SQLi) patterns such as ' OR '1'='1, UNION SELECT, and DROP TABLE.
- **Log Processing:** Extracts IP addresses and filters entries to focus on traffic originating from the local subnet (10.0.2.x).
- **Performance Optimization:** Used Python's set() data structure to maintain a collection of unique IP addresses, ensuring efficient and duplicate-free reporting.

Analysis Outcomes

- **Distinct IP Addresses:** Identified 4 unique IPs within the log dataset.
- **Security Findings:** Detected two significant malicious activities:
  - **Attack Case 1:** Login bypass attempt on login.php using the ' OR '1'='1 SQL injection technique.
  - **Attack Case 2:** Unauthorized data retrieval attempt on data.php using a UNION SELECT query.

Core Competencies Demonstrated

- **Log Investigation:** Ability to analyze raw server logs and identify abnormal or suspicious patterns.
- **Security Automation:** Experience in building Python scripts for automated threat detection.
- **Regex Expertise:** Strong use of pattern matching techniques to detect attack signatures.
- **Threat Attribution:** Ability to trace and associate malicious requests with their source IP addresses.